

LAPORAN TUGAS BESAR TEORI BAHASA FORMAL DAN AUTOMATA

Milestone 3 - Semantic Analysis



Oleh

13524014 - Yusuf Faishal Listyardi

13524046 - Farrell Limjaya

13524066 - Nathanael Gunawan

13524070 - A. Fawwaz Azam Wicaksono

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO & INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

2026

DAFTAR ISI

BAB I	
PENDAHULUAN.....	1
1.1. Pengantar.....	1
1.2. Spesifikasi.....	2
1.2.1. Revisi Program Milestone II.....	2
1.2.2. Spesifikasi Program.....	3
1.2.3. Predefined Identifier.....	7
1.2.4. Type Compatibility.....	7
BAB II	
DASAR TEORI.....	11
2.1. Semantic Analysis.....	11
2.2. Abstract Syntax Tree.....	12
2.3. Decorated Abstract Syntax Tree.....	12
2.4. Syntax-Directed Translation.....	13
2.5. Attributed Grammar.....	14
2.6. Symbol Table.....	15
2.7. Scope Resolution.....	16
2.8. Type Checking dan Type Compatibility.....	16
BAB III	
PERANCANGAN DAN IMPLEMENTASI.....	19
3.1. Gambaran Umum Program.....	19
3.2. Arsitektur Program.....	20
3.3. Representasi Abstract Syntax Tree.....	21
3.4. Transformasi Parse Tree ke AST.....	23
3.5. Perancangan Symbol Table.....	27
3.6. Implementasi Semantic Analyzer.....	30
3.7. Error Handling dan Diagnostic.....	32
BAB IV	
PENGUJIAN.....	34
4.1. Pengujian 1.....	34
4.2. Pengujian 2.....	35
4.3. Pengujian 3.....	39
4.4. Pengujian 4.....	42
4.5. Pengujian 5.....	44
4.6. Pengujian 6.....	45
4.7. Pengujian 7.....	47
4.8. Pengujian 8.....	48
BAB V	
KESIMPULAN DAN SARAN.....	51
5.1. Kesimpulan.....	51
5.2. Saran.....	52
BAB VI	
LAMPIRAN.....	53
6.1. Lampiran Link Repository Github.....	53
6.2. Pembagian Tugas.....	53
REFERENSI.....	54

BAB I

PENDAHULUAN

1.1. Pengantar

Pada milestone I lexical analysis, Anda berhasil mengubah source code menjadi bentuk token-token kecil dengan aturan DFA. Pada milestone II parser, dalam proses pengecekan syntax dengan Recursive Descent, Anda berhasil mengubah token-token tersebut menjadi sebuah parse tree.

Namun perlu diperhatikan bahwa sekedar pengecekan syntax tidak cukup untuk memastikan kebenaran dari program. Oleh karena itu, kita perlu melakukan pengecekan makna (semantic) dari program, yaitu pada tahapan Semantic Analysis.

Pengecekan semantic yang dilakukan oleh Semantic Analysis terdiri dari:

- **Type Checking (Pengecekan Tipe)**
Memastikan operator dan operand memiliki tipe yang kompatibel. Contoh: tidak menjumlahkan string dengan integer, perbandingan char terhadap string.
- **Symbol Table Management (Pengecekan Deklarasi)**
Memastikan setiap identifier (variabel/fungsi) telah dideklarasikan sebelum digunakan dan tidak dideklarasikan ulang dalam cakupan yang sama.
- **Scope Resolution (Pengecekan Lingkup)**
Menentukan validitas akses variabel berdasarkan hierarki blok kode. Contoh: scope global vs scope local.
- **Control Flow Validation (Pengecekan Alur Kontrol)**
Memastikan struktur alur kontrol yang logis. Contoh: instruksi break/continue hanya dalam loop, fungsi dipastikan memiliki return pada semua cabang kontrol, warning untuk unreachable code.

Semantic Analysis akan melakukan anotasi terhadap parse tree yang telah dihasilkan oleh parser dan menghasilkan sebuah decorated abstract syntax tree (AST). Anotasi berupa

(dan tak terbatas kepada) penambahan informasi, seperti tipe data dan referensi simbol (identifier) ke symbol table.

Decorated Abstract Syntax Tree (Decorated AST)

Semantic Analysis akan melakukan anotasi terhadap AST hasil konversi dari parse tree untuk menghasilkan sebuah Decorated AST. Anotasi yang dilakukan bertujuan memberikan konteks untuk pengecekan semantic, seperti penambahan simbol table, tipe data, penambahan reserve keywords, pelacakan identifier, penanda tingkatan scope, dan sebagainya. Implementasi Decorated AST memang beragam, namun dalam tugas ini, kita akan berbasis 3 symbol table, yaitu tab, btab, dan atab yang dijelaskan dalam spesifikasi tugas. Jadi Decorated AST merupakan hasil dari proses pengecekan semantic pada Semantic Analysis.

Berikut adalah beberapa istilah yang perlu dipahami:

1. **Attributed Grammar**

Grammar yang dilengkapi dengan atribut dan aturan semantik.

2. **Symbol Table**

Struktur data yang minimal menyimpan informasi nama identifier, tipe datanya dan scopenya. Symbol table menggunakan struktur data stack.

3. **Visitor / Visit Functions**

Untuk setiap jenis node akan memiliki fungsi visit_<node> yang akan:

1. Memanggil visit pada child-nya
2. Mengakses atau memperbarui symbol table
3. Menghitung atribut/tipe semantik
4. Menambahkan anotasi pada node

1.2. Spesifikasi

1.2.1. Revisi Program Milestone II

Untuk pengerjaan milestone III, terdapat beberapa revisi yang diterapkan:

1. Penggunaan token pada longest match bersifat opsional, namun lebih baik diterapkan untuk meningkatkan keterbacaan.

2. Untuk NonSymbol yang berdekatan dan beda tipe (contoh: 5e, x2) harus dipisah.
3. Comment tidak perlu dijadikan node dalam parse tree.
4. Statement bisa dalam bentuk empty statement, sehingga
begin
;
end
Dianggap valid

Selain itu terdapat beberapa perubahan pada node:

1. `<while-statement>` $\rightarrow$ `whilesy + expression + dosy + compound-statement + semicolon`
2. `<for-statement>` $\rightarrow$ `forsy + ident + becomes + expression + ( tosy | downtosy) + expression + dosy + compound-statement + semicolon`

1.2.2. Spesifikasi Program

Pada milestone ini, Anda diminta untuk membuat tahapan ketiga dari compiler, yaitu semantic analysis. Tahapan ini berfungsi melakukan analisis makna (semantic analysis) dengan menggunakan Attributed Grammar.

Parse tree yang sudah dihasilkan akan ditelusuri secara top-down menggunakan fungsi visit (semantic visitor) pada setiap node-nya yang dibuat berdasarkan grammar yang sebelumnya didefinisikan. Selama proses ini, symbol table digunakan untuk menyimpan dan mengambil informasi deklarasi simbol/identifier.

Masukan	Parse Tree
Keluaran	Decorated AST, Symbol Table
Algoritma	L-Attributed Grammar

Pada Semantic Analysis, akan dilakukan beberapa tahapan berikut:

1. Konversi Parse Tree menjadi AST dengan Syntax-Directed Translation:

- a. Memanfaatkan Syntax-Directed Translation Scheme untuk menyisipkan semantic action/rule pada aturan produksi grammar. Semantic Action berfungsi untuk mengevaluasi nilai atribut dari masing-masing node. Tuliskan semantic action yang Anda gunakan pada laporan.
- b. Menjalankan action pada setiap node parse tree untuk membangun struktur AST yang ringkas dengan cara:
 - i. Membentuk node baru untuk AST (seperti AssignNode, BinOpNode) yang merepresentasikan hubungan semantik antar operator dan operand.
 - ii. Menghilangkan informasi sintaksis yang sudah tidak diperlukan untuk tahap analisis (seperti tanda kurung berlebih, keyword terminal BEGIN, END, dan SEMICOLON).
 - iii. Menggabungkan potongan node dari bawah ke atas dengan node level bawah akan diserahkan ke node induknya untuk dirakit menjadi satu kesatuan AST yang utuh.

2. Proses Dekorasi AST (Type & Scope Checking)

Penelusuran secara Depth-First Search (DFS) menggunakan fungsi visit:

- a. Mengelola scope dengan melakukan Push pada stack Symbol Table (tabel tab dan btab) saat memasuki blok baru (seperti fungsi, prosedur, atau record) dan melakukan Pop saat keluar dari blok tersebut.
- b. Melakukan registrasi simbol dengan memasukkan detail identifier seperti nama, jenis objek, tipe data, level leksikal, dan alamat memori ke dalam tabel tab setiap kali ditemukan deklarasi.
- c. Melakukan lookup pada Symbol Table dari tingkat scope terdalam hingga terluar setiap kali menemukan penggunaan identifier untuk memastikan bahwa variabel atau fungsi tersebut sudah dideklarasikan sebelumnya.
- d. Melakukan inferensi dan validasi tipe data dengan langkah-langkah:

- i. Mengkalkulasi tipe data hasil dari setiap operasi pada node (misalnya, operasi dua operand Integer akan menghasilkan tipe Integer).
 - ii. Memastikan kompatibilitas tipe data (berdasarkan aturan Type Compatibility) antara variabel tujuan dengan nilai ekspresi yang diberikan.
- e. Anotasi setiap node AST dengan informasi tambahan:
- i. Tipe data akhir dari ekspresi tersebut.
 - ii. Referensi atau indeks yang mengarah pada entri yang tepat di Symbol Table.
 - iii. Tingkatan lexical level atau scope tempat node tersebut berada.

Sebelum proses dimulai, lakukan inisiasi symbol table yang terdiri dari 3 jenis:

1. tab: digunakan untuk menyimpan informasi mengenai identifier termasuk konstanta, variabel, prosedur, atau fungsi.
2. btab: digunakan untuk menyimpan informasi blok prosedur dan definisi tipe rekord.
3. atab: digunakan untuk menyimpan informasi array seperti tipe index, batasan index, tipe elemen, dan ukuran array.

Berikut merupakan daftar atribut dari symbol table:

tab	
atribut	deskripsi
identifiers	Nama identifier (misalnya nama variabel, konstanta, tipe, prosedur, fungsi). Indeks dimulai dari 33 karena ada reserved words termasuk predefined identifiers.
link	Pointer/indeks ke identifier sebelumnya dalam scope yang sama. Digunakan untuk manajemen scope (linked list per blok).
obj	Kelas objek yang dienumerasi: konstanta, variabel, tipe, prosedur, fungsi, dll.

type	Tipe dasar dari identifier, misalnya: integer, boolean, char, real, array, record, dll. Biasanya berupa kode numerik.
ref	Pointer/indeks ke tabel lain jika tipe adalah komposit (array/record). Mengarah ke atab (array table) atau btab (record/procedure block).
nrm	Menandai apakah identifier adalah variabel normal (=1) atau parameter by-reference (var parameter) (=0).
lev	Tingkat <i>lexical level</i> tempat identifier dideklarasikan (0 = global, 1 = dalam prosedur, 2 = dalam prosedur di dalam prosedur, dst).
adr	Makna tergantung jenis objek: offset variabel di stack frame, nilai konstanta, offset field record, alamat prosedur, atau ukuran/penanda lain.

btab	
atribut	deskripsi
blocks	Indeks entri block (setiap block mewakili prosedur, fungsi, atau record type definition).
last	Pointer/indeks ke identifier terakhir yang dideklarasikan di dalam block tersebut (menghubungkan field record, parameter, atau variabel lokal).
lpar	Pointer/indeks ke parameter terakhir dari prosedur/fungsi pada block tersebut. Jika block adalah record, nilainya 0.
psze	Total ukuran parameter block (dalam byte/unit memori).
vsze	Total ukuran variabel lokal block (dalam byte/unit memori)

Dengan memiliki informasi pada symbol table, dapat dipastikan konsistensi tipe data serta memastikan variabel tidak bisa dioperasikan jika belum diinisialisasi dengan nilai.

1.2.3. Predefined Identifier

Terdapat beberapa ident yang harus sudah terdefinisi di dalam symbol table, sebagai berikut:

Identifier	Keterangan
Real	Memiliki type Real e.g. someArray[1..10] of Real
Integer	Memiliki type Integer e.g. procedure print(x: integer);

Char	Memiliki type Char e.g. var a: Char
Boolean	Memiliki type Boolean e.g. var b: Boolean
String	Memiliki type String e.g. var c: String
True	Memiliki type Boolean dengan nilai <i>true</i> e.g. const isTrue == true
False	Memiliki type Boolean dengan nilai <i>false</i> e.g. if (someVar = false) then

Anda boleh menambahkan predefined identifier lain jika diperlukan (e.g. fungsi Readln, Writeln). **Tuliskan predefined Identifier yang Anda gunakan pada laporan.**

1.2.4. Type Compatibility

Suatu “type” menentukan “bentuk data” yang dapat ditampung constant (variable), expression, functions, dll. Type Compatibility Rule menentukan bagaimana type dapat digunakan di dalam expression, statements (e.g. assignments), dll.

Pada Milestone ini, Anda diminta untuk menentukan Type Compatibility Rule yang akan dipakai di bahasa Arion. Hal ini mencakup:

1. Menentukan aturan khusus mengenai type yang harus digunakan oleh suatu Node (e.g. expression di dalam if-statement harus bertipe Boolean);
2. Aturan interaksi antara dua type (e.g. constant di dalam range harus memiliki type yang sama).
3. Semantic Rule yang berhubungan dengan type (e.g. constant yang diproduksi dari realcon memiliki type Real/constant_real)

Terdapat beberapa type dan rule dasar yang harus Anda definisikan. Type dan rule dasar berikut dapat Anda tambahkan dan kembangkan lebih lanjut. **Tuliskan Type Compatibility Rule yang Anda gunakan di dalam Laporan.**

Type dasar, *simple type*, yang digunakan dalam bahasa pemrograman Arion adalah sebagai berikut:

No	Nama Type	Keterangan
	Real	realcon memiliki type Real
	Integer	intcon memiliki type Integer
	Char	charcon memiliki type Char

	Boolean	term dan expression yang diproduksi dari notsy, andsy, dan orsy memiliki type Boolean expression yang diproduksi dari relational-operator memiliki type Boolean
	String	string memiliki type String
	Subrange	Type dari Node Range ditentukan oleh type dari Constant di dalamnya. Lower Bound dari Range (Constant di kiri) tidak boleh lebih besar daripada Upper Bound dari Range. Subrange tidak boleh memiliki type Real.
	Enumerated	Type dari Node Enumerated ditentukan oleh type dari Ident di dalamnya. Ident di dalam enumerated harus memiliki type yang sama.

Terdapat juga structured type dengan contoh aturan sebagai berikut:

Nama Type	Keterangan
Array	Array memiliki struktur berikut: array [T1] of T2 Dengan T1 adalah index type dan T2 adalah element type. Index type harus memiliki <i>simple type</i> bukan Real. Sedangkan T2 dapat berupa type apapun.
Record	Field di dalam Record dapat memiliki type apapun

Structured type dapat bersifat *named*, jika didefinisikan di dalam type-declaration, atau *anonymous*, jika didefinisikan secara langsung (e.g. di dalam var-declaration). Type dari *named* record ditentukan berdasarkan nama ident di dalam type-declaration. Dua *anonymous* record, meskipun memiliki struktur yang sama, bersifat *incompatible*.

E.g.:

```
var
  point1: record x, y: Integer; end;
  point2: record x, y: Integer; end;

begin
  point1.x := 10;
  {Incompatible Error}
  point2 := point1;
end;
```

Adapun Type Compatibility Rule dasar sebagai berikut:

- a. Dua type adalah *compatible* jika memenuhi salah satu dari kondisi berikut:
 1. Kedua type memiliki type yang sama
 2. Satu type adalah subrange dari yang lain (e.g. [1..10] dengan Integer 5), atau kedua type adalah subrange dari type yang sama (e.g. [1..10] dengan [50..100]).
 3. Kedua type memiliki type String dan memiliki panjang yang sama
- b. Sebuah type T2 adalah *assignment-compatible* dengan type T1 jika memenuhi salah satu dari kondisi berikut:
 - i. T1 adalah Real dan T2 adalah Integer
 - ii. T1 dan T2 adalah *compatible*, bertipe Integer, Boolean, atau Char, dan nilai (value) yang dimiliki T2 adalah subset dari nilai yang dimiliki T1 (e.g. T2 = 5 dan T1 = [1..10])
 - iii. determined by T 1.
 - iv. T1 and T2 adalah *compatible* dan bertipe String
- c. Suatu pasangan ident dan expression dapat digunakan dalam assignment-statement jika ident dan expression tersebut adalah *assignment-compatible*, sebagai contoh:

```
var
  index: 1..10;
  count: Integer;
begin
  count := 5;
  index := count;

  { index := 11; } { Assignment-Incompatible Error }
end;
```

BAB II

DASAR TEORI

2.1. Semantic Analysis

Semantic analysis merupakan tahap ketiga dalam proses kompilasi yang dilakukan setelah lexical analysis dan syntax analysis. Pada tahap lexical analysis, source code diubah menjadi token-token. Selanjutnya, syntax analysis memeriksa apakah susunan token tersebut sesuai dengan grammar bahasa pemrograman dan membentuk struktur seperti parse tree. Namun, program yang valid secara sintaks belum tentu memiliki makna yang valid. Oleh karena itu, semantic analysis diperlukan untuk memeriksa kebenaran program berdasarkan aturan semantik bahasa pemrograman.

Jika syntax analysis berfokus pada struktur program, semantic analysis berfokus pada makna dari struktur tersebut. Sebagai contoh, suatu assignment dapat memiliki bentuk yang benar menurut grammar, tetapi tetap salah apabila nilai yang diberikan tidak sesuai dengan tipe variabel tujuan.

Pada Milestone 3, semantic analysis digunakan untuk melakukan beberapa pemeriksaan utama, yaitu:

1. Pemeriksaan deklarasi identifier, seperti variabel, konstanta, tipe, procedure, dan function.
2. Pemeriksaan scope untuk memastikan identifier diakses pada lingkup yang sesuai.
3. Pemeriksaan kesesuaian tipe data pada ekspresi, assignment, operator, dan pemanggilan procedure atau function.
4. Pemeriksaan penggunaan tipe terstruktur seperti array dan record.
5. Pemberian anotasi semantik pada struktur program hasil parsing.

Hasil dari semantic analysis tidak hanya berupa status apakah program valid atau tidak valid secara semantik, tetapi juga berupa symbol table, decorated Abstract Syntax Tree, serta pesan diagnostic apabila ditemukan kesalahan.

2.2. Abstract Syntax Tree

Abstract Syntax Tree (AST) adalah representasi struktur program dalam bentuk pohon. Source code pada dasarnya ditulis sebagai teks yang bersifat linear, sedangkan struktur program umumnya bersifat hierarkis dan rekursif. Ekspresi dapat memuat subekspresi lain, statement dapat berada di dalam block, dan struktur seperti percabangan maupun perulangan dapat memuat statement lain di dalamnya. Oleh karena itu, struktur pohon lebih sesuai untuk merepresentasikan hubungan antarbagian program.

Dalam AST, setiap node merepresentasikan konstruksi penting pada program. Sebagai contoh, literal dapat direpresentasikan sebagai node daun, sedangkan operasi biner dapat direpresentasikan sebagai node yang menyimpan operator serta dua operand, yaitu operand kiri dan operand kanan. Karena operand tersebut juga dapat berupa ekspresi lain, AST mampu merepresentasikan ekspresi bertingkat secara rekursif.

AST berbeda dari parse tree. Parse tree masih mempertahankan detail grammar dan token sintaksis secara lengkap, sedangkan AST menyimpan informasi yang lebih relevan untuk tahap analisis berikutnya. Detail seperti tanda kurung berlebih, keyword pembatas tertentu, atau token semicolon dapat dihilangkan apabila tidak diperlukan untuk memahami makna struktur program.

Pada Milestone 3, AST dibentuk dari parse tree hasil syntax analysis. Struktur ini digunakan sebagai dasar semantic analysis karena hubungan antara deklarasi, statement, ekspresi, operator, operand, pemanggilan procedure atau function, serta akses array dan record dapat diproses dengan lebih langsung dibandingkan melalui parse tree.

2.3. Decorated Abstract Syntax Tree

Decorated Abstract Syntax Tree adalah AST yang telah diberi informasi tambahan hasil semantic analysis. Informasi tambahan tersebut disebut anotasi semantik. Anotasi ini diperlukan agar setiap node tidak hanya menunjukkan struktur program, tetapi juga menyimpan konteks semantik yang berkaitan dengan tipe data, identifier, dan scope.

Beberapa contoh anotasi yang dapat diberikan pada node AST adalah:

1. Tipe data hasil evaluasi suatu ekspresi.
2. Indeks atau referensi menuju entri pada symbol table.

3. Lexical level atau level scope tempat identifier berada.
4. Referensi terhadap tipe terstruktur seperti array atau record.

Sebagai contoh, node variabel *a* pada AST dapat diberi anotasi bahwa *a* bertipe Integer, terdaftar pada indeks tertentu di symbol table, dan dideklarasikan pada scope global. Begitu pula node operasi biner seperti $b + 10$ dapat diberi anotasi bahwa hasil operasinya bertipe Integer setelah kedua operand dinyatakan kompatibel.

Decorated AST menjadi keluaran penting pada Milestone 3 karena menunjukkan hasil analisis semantik yang telah dilakukan terhadap struktur program. Representasi ini juga dapat menjadi dasar bagi tahap kompilasi berikutnya, seperti intermediate code generation atau code generation.

2.4. Syntax-Directed Translation

Syntax-Directed Translation (SDT) adalah metode penerjemahan struktur program dengan mengaitkan aturan grammar dan semantic action. Pada pendekatan ini, setiap aturan produksi dapat memiliki aksi tertentu yang dijalankan untuk menghasilkan informasi atau representasi baru selama struktur sintaks diproses.

Dalam proses kompilasi, SDT dapat digunakan untuk berbagai kebutuhan, seperti evaluasi ekspresi, pemeriksaan tipe, pembentukan intermediate representation, maupun pembentukan struktur data program. Pada Milestone 3, konsep SDT digunakan terutama untuk mengubah parse tree hasil syntax analysis menjadi Abstract Syntax Tree (AST).

Proses pembentukan AST dilakukan dengan memanfaatkan struktur parse tree. Ketika node-node parse tree dikunjungi, semantic action dapat membentuk node AST baru, meneruskan hasil child ke parent, atau menggabungkan beberapa node menjadi struktur yang lebih ringkas. Dengan demikian, informasi sintaksis yang tidak lagi diperlukan dapat dihilangkan, sedangkan hubungan semantik penting tetap dipertahankan.

Sebagai contoh, pada suatu assignment, bagian identifier dan expression dapat diterjemahkan menjadi node assignment yang memiliki target dan value. Pada ekspresi dengan operator biner, operand kiri dan operand kanan dapat digabungkan menjadi node operasi biner yang menyimpan operator serta kedua operand tersebut.

SDT juga memanfaatkan atribut untuk membawa informasi selama proses translasi. Atribut dapat menyimpan nilai literal, tipe data, node AST sementara, atau informasi lain yang dibutuhkan oleh semantic action. Dengan atribut tersebut, hasil pemrosesan dari node yang lebih kecil dapat digunakan untuk membangun node yang lebih besar.

Melalui Syntax-Directed Translation, konversi dari parse tree menuju AST dapat dilakukan secara sistematis sesuai struktur grammar yang telah dikenali oleh parser. Hasil AST tersebut kemudian menjadi dasar bagi semantic analyzer untuk melakukan pemeriksaan tipe, scope, identifier, dan aturan semantik lainnya.

2.5. Attributed Grammar

Attributed grammar adalah context-free grammar yang diperluas dengan atribut dan aturan semantik. Context-free grammar hanya menjelaskan bentuk sintaks suatu bahasa, sedangkan attributed grammar memungkinkan informasi semantik ikut dihitung ketika struktur program diproses.

Atribut digunakan untuk membawa informasi tambahan pada simbol grammar. Informasi tersebut dapat berupa tipe data suatu ekspresi, nilai konstanta, informasi scope identifier, maupun node AST yang sedang dibentuk. Nilai atribut dihitung melalui semantic rules yang dikaitkan dengan aturan produksi grammar.

Secara umum, atribut dapat dibedakan menjadi:

1. Synthesized attribute, yaitu atribut yang dihitung dari child dan diteruskan ke parent.
2. Inherited attribute, yaitu atribut yang diperoleh dari parent atau sibling untuk digunakan oleh suatu node.

Pada Milestone 3, konsep attributed grammar digunakan sebagai dasar untuk menghubungkan struktur grammar dengan informasi semantik yang diperlukan pada proses analisis. Ketika parse tree dikonversi menjadi AST, atribut dapat membawa node-node AST yang dibentuk dari bagian bawah tree. Setelah itu, ketika AST dianalisis, informasi seperti tipe data, lexical level, dan referensi symbol table dapat dihitung dan ditambahkan sebagai anotasi.

Spesifikasi Milestone 3 menggunakan pendekatan L-Attributed Grammar. Pada pendekatan ini, atribut dapat dievaluasi secara terarah dari kiri ke kanan dengan memanfaatkan atribut synthesized dan inherited yang tersedia saat node diproses. Pendekatan ini sesuai untuk proses semantic analysis karena informasi semantik dapat dihitung berdasarkan struktur program sekaligus konteks yang telah diketahui sebelumnya.

2.6. Symbol Table

Symbol table adalah struktur data yang digunakan semantic analyzer untuk menyimpan informasi mengenai identifier dalam program. Identifier yang disimpan dapat berupa variabel, konstanta, tipe, procedure, function, parameter, dan nama program.

Informasi yang disimpan pada symbol table dapat meliputi:

1. Nama identifier.
2. Jenis objek identifier.
3. Tipe data identifier.
4. Scope atau lexical level.
5. Referensi terhadap tipe komposit.
6. Informasi alamat, offset, atau ukuran data.

Pada Milestone 3, symbol table terdiri atas tiga bagian utama, yaitu:

1. tab, yaitu tabel utama untuk menyimpan identifier seperti variabel, konstanta, tipe, procedure, dan function.
2. btab, yaitu tabel yang menyimpan informasi block, procedure, function, atau record.
3. atab, yaitu tabel yang menyimpan informasi tipe array, seperti tipe indeks, batas bawah, batas atas, tipe elemen, dan ukuran array.

Symbol table digunakan ketika semantic analyzer menemukan deklarasi maupun penggunaan identifier. Saat deklarasi ditemukan, identifier baru dimasukkan ke dalam symbol table. Saat identifier digunakan, analyzer melakukan lookup untuk memastikan identifier tersebut sudah dideklarasikan dan digunakan sesuai jenis serta tipenya.

2.7. Scope Resolution

Scope adalah wilayah program tempat suatu identifier dapat dikenali dan digunakan. Dalam bahasa pemrograman, scope penting karena identifier dengan nama yang sama dapat muncul pada block yang berbeda, dan tidak semua identifier boleh diakses dari semua lokasi program.

Semantic analyzer perlu mengelola scope ketika memasuki atau keluar dari suatu block, procedure, function, maupun record. Saat memasuki scope baru, symbol table dapat menambahkan konteks baru. Saat keluar dari scope tersebut, analyzer kembali ke konteks sebelumnya.

Lookup identifier dilakukan dari scope terdalam menuju scope yang lebih luar. Dengan cara ini, semantic analyzer dapat:

1. Menentukan deklarasi mana yang dirujuk oleh suatu identifier.
2. Menemukan identifier yang digunakan sebelum dideklarasikan.
3. Mendeteksi redeklarasikan identifier pada scope yang sama.
4. Memisahkan identifier lokal dari identifier global.

Sebagai contoh, variabel lokal pada procedure hanya dapat digunakan selama masih berada dalam lingkup procedure tersebut, sedangkan variabel global dapat diakses dari lingkup yang lebih luas selama tidak tertutup oleh deklarasi lokal lain.

2.8. Type Checking dan Type Compatibility

Type checking adalah proses pemeriksaan tipe data pada suatu program agar setiap nilai, ekspresi, operator, dan statement digunakan sesuai aturan tipe bahasa pemrograman. Pada compiler, type checking termasuk bagian penting dari semantic analysis karena suatu program yang benar secara sintaks belum tentu menggunakan tipe data secara benar.

Secara umum, type checking dapat dibedakan menjadi dua jenis, yaitu:

1. Static Type Checking

Static type checking adalah pemeriksaan tipe yang dilakukan pada saat compile time. Compiler memeriksa tipe berdasarkan struktur program, deklarasi identifier, dan aturan semantik bahasa sebelum program dijalankan.

Pada Milestone 3, pemeriksaan yang dilakukan termasuk ke dalam static type checking karena semantic analyzer memeriksa tipe pada AST dengan bantuan symbol table sebelum tahap eksekusi program.

2. Dynamic Type Checking

Dynamic type checking adalah pemeriksaan tipe yang dilakukan ketika program sedang berjalan. Pada pendekatan ini, tipe suatu nilai diperiksa pada runtime. Dynamic type checking umumnya digunakan pada bahasa yang lebih fleksibel dalam penggunaan tipe.

Milestone 3 berfokus pada semantic analysis saat compile time, sehingga pembahasan utama pada laporan ini adalah static type checking.

Dalam semantic analysis, type checking dapat digunakan untuk melakukan beberapa pemeriksaan berikut:

1. Memastikan operator digunakan pada operand dengan tipe yang sesuai.
2. Memastikan nilai pada assignment sesuai dengan tipe variabel tujuan.
3. Memastikan kondisi pada statement seperti if, while, dan repeat-until menghasilkan tipe Boolean.
4. Memastikan pemanggilan function atau procedure menggunakan jumlah dan tipe argumen yang sesuai.
5. Memastikan operasi pada tipe terstruktur dilakukan secara valid, misalnya indexing hanya dilakukan pada array dan field access hanya dilakukan pada record.
6. Memastikan tipe hasil suatu ekspresi dapat ditentukan berdasarkan operand dan operator yang digunakan.

Pada bahasa Arion, tipe dasar yang digunakan mencakup:

1. Integer.
2. Real.
3. Char.
4. Boolean.
5. String.

Selain itu, terdapat pula tipe lain seperti subrange, enumerated type, array, dan record. Setiap tipe memiliki aturan penggunaan masing-masing. Sebagai contoh:

1. Operator aritmetika memerlukan operand numerik.
2. Operator logika seperti and, or, dan not memerlukan operand Boolean.
3. Kondisi pada if, while, dan repeat-until harus menghasilkan tipe Boolean.
4. Index array harus sesuai dengan tipe indeks array.
5. Field pada record harus benar-benar terdefinisi pada record tersebut.

Type checking berkaitan erat dengan type compatibility. Type compatibility adalah aturan yang menentukan apakah dua tipe dapat digunakan bersama pada suatu operasi atau konteks tertentu. Dalam assignment, aturan yang digunakan disebut assignment compatibility, yaitu aturan untuk menentukan apakah nilai di sisi kanan dapat diberikan kepada target di sisi kiri.

Pada beberapa kasus, tipe Integer dapat diberikan kepada variabel bertipe Real karena nilai Integer masih dapat direpresentasikan sebagai Real. Namun, assignment pada subrange harus memperhatikan batas nilai yang diperbolehkan. Dengan demikian, semantic analyzer tidak hanya memeriksa nama tipe, tetapi juga aturan makna dari tipe tersebut.

Melalui type checking dan type compatibility, semantic analyzer dapat mendeteksi kesalahan seperti:

1. Menjumlahkan nilai String dengan Integer.
2. Menggunakan ekspresi Integer sebagai kondisi if.
3. Memberikan nilai dengan tipe yang tidak cocok pada assignment.
4. Mengakses array menggunakan indeks dengan tipe yang salah.
5. Mengakses field record yang tidak tersedia.

BAB III

PERANCANGAN DAN IMPLEMENTASI

Implementasi semantic analyzer pada Milestone 3 dibangun di atas hasil lexical analyzer dan syntax analyzer yang telah diimplementasikan pada milestone sebelumnya. Lexer tetap bertugas mengubah source code Arion menjadi deretan token, sedangkan parser bertugas memeriksa kesesuaian token terhadap grammar bahasa Arion dan membentuk parse tree. Pada Milestone 3, parse tree tersebut kemudian ditransformasikan menjadi Abstract Syntax Tree (AST) agar struktur program menjadi lebih ringkas dan lebih sesuai untuk proses analisis semantik.

Semantic analyzer bertugas memeriksa kebenaran makna program yang tidak dapat sepenuhnya diperiksa pada tahap sintaks, seperti deklarasi identifier, cakupan scope, kesesuaian tipe data, assignment, pemanggilan procedure dan function, penggunaan array dan record, serta validitas ekspresi dan statement. Selama proses analisis, program kami membangun symbol table yang terdiri atas tab, btab, dan atab, lalu menambahkan anotasi tipe, indeks symbol table, dan level scope pada node AST. Hasil akhir Milestone 3 berupa decorated AST, tabel simbol, diagnostic message, serta status analisis semantik.

3.1. Gambaran Umum Program

Pada Milestone 3, program dikembangkan dengan menambahkan tahap semantic analysis setelah proses lexical analysis dan syntax analysis. Tahap ini bertujuan untuk memastikan bahwa program tidak hanya benar secara sintaks, tetapi juga valid secara semantik. Dengan demikian, source code Arion yang telah berhasil diproses oleh lexer dan parser akan dianalisis lebih lanjut untuk memeriksa penggunaan identifier, deklarasi variabel dan tipe, kesesuaian tipe data, cakupan scope, pemanggilan procedure dan function, serta penggunaan struktur data seperti array dan record.

Secara umum, alur kerja program dimulai dari pembacaan source code Arion sebagai input. Source code tersebut diproses oleh lexer untuk menghasilkan deretan token. Token komentar diabaikan karena tidak mempengaruhi struktur maupun makna program, sedangkan token tidak dikenal akan menghasilkan lexical error. Deretan token yang valid kemudian diproses oleh parser untuk menghasilkan parse tree. Parse tree ini selanjutnya

dikonversi menjadi Abstract Syntax Tree (AST) agar struktur program lebih ringkas dan lebih mudah dianalisis pada tahap semantik.

Setelah AST terbentuk, semantic analyzer melakukan traversal terhadap node-node AST menggunakan pendekatan visitor. Pada tahap ini, analyzer membangun symbol table untuk mencatat informasi identifier, block, array, tipe data, parameter, procedure, function, dan variabel. Selain itu, setiap node yang relevan pada AST diberi anotasi berupa informasi tipe, indeks symbol table, dan level scope. Apabila ditemukan pelanggaran semantik, seperti identifier yang belum dideklarasikan, redeklarasikan dalam scope yang sama, assignment dengan tipe yang tidak sesuai, atau pemanggilan procedure dengan argumen yang salah, analyzer akan menghasilkan diagnostic message yang memuat informasi lokasi dan penyebab kesalahan.

Keluaran utama program pada Milestone 3 terdiri atas symbol table, decorated AST, semantic diagnostics, dan status akhir analisis. Jika tidak ditemukan kesalahan semantik, program akan menghasilkan status SUCCESS. Sebaliknya, jika terdapat satu atau lebih kesalahan semantik, program akan menghasilkan status FAILED.

3.2. Arsitektur Program

Pada Milestone 3, arsitektur program ditambahkan dengan beberapa modul yaitu pembentuk AST, semantic analyzer, symbol table, serta modul integrasi input-output.

Sebelum dianalisis, parse tree yang dihasilkan dari milestone 2 dikonversi terlebih dahulu menjadi Abstract Syntax Tree melalui modul AST. AST menyederhanakan struktur parse tree dengan hanya mempertahankan elemen-elemen penting program, seperti deklarasi, tipe data, statement, ekspresi, pemanggilan procedure/function, akses array, dan akses record. Representasi ini membuat proses semantic analysis lebih terarah karena analyzer tidak perlu menangani seluruh detail token dan aturan produksi grammar.

Tahap semantic analysis dijalankan oleh kelas SemanticAnalyzer. SemanticAnalyzer melakukan traversal terhadap AST dengan fungsi visitor yang dibagi berdasarkan kategori node, seperti deklarasi, tipe, ekspresi, dan statement. Dalam proses traversal tersebut, semantic analyzer membangun dan memanfaatkan symbol table untuk menyimpan informasi identifier, tipe, scope, parameter, procedure, function, array, dan record. Selain itu, semantic analyzer juga menambahkan anotasi pada AST berupa tipe

hasil inferensi, indeks symbol table, dan lexical level sehingga AST akhir menjadi decorated AST.

Symbol table pada program kami direpresentasikan melalui tiga tabel utama, yaitu tab, btab, dan atab. Tabel tab menyimpan informasi identifier, tabel btab menyimpan informasi block atau scope, sedangkan tabel atab menyimpan informasi tipe array. Ketiga tabel ini digunakan bersama oleh semantic analyzer untuk melakukan lookup identifier, memeriksa redeklarasasi, menentukan ukuran tipe, memvalidasi parameter, serta mengecek kompatibilitas tipe.

Integrasi seluruh tahapan dilakukan melalui modul driver dan cli. Modul cli menangani interaksi dengan pengguna, pemilihan folder test, pembacaan file input, dan penyimpanan file output. Modul driver menentukan analyzer yang dijalankan sesuai milestone yang dipilih. Untuk Milestone 3, driver menjalankan proses dari source code atau parse tree menuju semantic analyzer, lalu menghasilkan output berupa symbol table, decorated AST, semantic diagnostics, dan status akhir analisis.

3.3. Representasi Abstract Syntax Tree

Pada Milestone 3, digunakan Abstract Syntax Tree (AST) sebagai representasi utama struktur program sebelum dilakukan semantic analysis. AST dibentuk dari parse tree yang dihasilkan parser. Berbeda dengan parse tree yang masih merepresentasikan aturan produksi grammar secara lengkap, AST hanya menyimpan bagian-bagian program yang penting untuk analisis semantik, seperti deklarasi, tipe data, statement, ekspresi, pemanggilan procedure dan function, serta akses terhadap array dan record. Dengan struktur yang lebih ringkas, AST membuat proses traversal dan pengecekan semantik menjadi lebih mudah dilakukan.

Representasi AST didefinisikan pada modul ast, terutama melalui kelas dasar ASTNode. Setiap node AST memiliki informasi jenis node, lokasi sumber program, tipe hasil inferensi, indeks symbol table, dan lexical level. Informasi lokasi digunakan untuk membantu pelaporan error, sedangkan informasi tipe, indeks symbol table, dan lexical level digunakan sebagai anotasi setelah proses semantic analysis. Dengan demikian, AST tidak hanya merepresentasikan bentuk program, tetapi juga dapat berkembang menjadi decorated AST setelah diberi informasi semantik.

Jenis node pada AST dibedakan menggunakan enumerasi `ASTNodeKind`. Enumerasi ini mencakup berbagai kategori struktur program Arion, seperti `Program`, `Block`, `VarDecl`, `ConstDecl`, `TypeDecl`, `ParamDecl`, `ProcDecl`, `FuncDecl`, `Assign`, `If`, `While`, `For`, `Repeat`, `Case`, `ProcCall`, `FuncCall`, `BinOp`, `UnaryOp`, `Var`, `literal`, `ArrayAccess`, dan `RecordAccess`. Pembagian ini membuat setiap bentuk konstruksi program dapat direpresentasikan secara eksplisit dan diproses sesuai kebutuhannya pada semantic analyzer.

AST pada program kami juga menggunakan beberapa turunan node yang dikelompokkan berdasarkan fungsinya. `TypeNode` digunakan untuk merepresentasikan tipe, seperti `named type`, `array type`, `record type`, `subrange type`, dan `enum type`. `DeclarationNode` digunakan untuk deklarasi variabel, konstanta, tipe, parameter, procedure, dan function. `StatementNode` digunakan untuk statement seperti `assignment`, `conditional`, `loop`, `case`, dan `procedure call`. Sementara itu, `ExpressionNode` digunakan untuk ekspresi seperti operasi biner, operasi unary, literal, variabel, function call, array access, dan record access.

Setelah AST terbentuk, setiap node dapat dicetak melalui method `toString()`. Method ini digunakan untuk menghasilkan representasi tree yang mudah dibaca. Pada tahap awal, AST hanya berisi struktur program. Setelah semantic analyzer berjalan, node-node AST akan diberi anotasi berupa tipe, indeks symbol table, dan lexical level. Hasil akhir inilah yang disebut decorated AST, yaitu AST yang tidak hanya menunjukkan struktur program, tetapi juga memuat informasi semantik hasil analisis.


```

1  struct ASTNode {
2      explicit ASTNode(ASTNodeKind kind);
3      virtual ~ASTNode() = default;
4
5      ASTNodeKind kind;
6      SourceLocation location;
7      int inferredType = 0;
8      int tabIndex = -1;
9      int lexLevel = 0;
10
11     virtual std::string toString(int indent = 0) const = 0;
12 };
13
14 struct TypeNode : ASTNode {
15     explicit TypeNode(ASTNodeKind kind);
16     int typeCode = 0;
17     int ref = 0;
18     bool isNamed = false;
19     std::string typeName;
20 };
21
22 struct NamedTypeNode : TypeNode {
23     std::string name;
24
25     NamedTypeNode();
26     std::string toString(int indent = 0) const override;
27 };

```

Gambar Sebagian Struktur Pembentukan AST

3.4. Transformasi Parse Tree ke AST

Transformasi dilakukan melalui fungsi `buildAST()` pada modul `ast`. Fungsi tersebut menerima root parse tree berupa `ParseNode`, kemudian membangun root AST berupa `ProgramNode`. Proses transformasi berjalan dengan membaca label-label parse tree, seperti `<program>`, `<declaration-part>`, `<compound-statement>`, `<expression>`, dan `<type>`, lalu mengubahnya menjadi node AST yang lebih spesifik. Sebagai contoh, deklarasi variabel pada parse tree dikonversi menjadi `VarDeclNode`, deklarasi konstanta menjadi `ConstDeclNode`, ekspresi operasi menjadi `BinOpNode` atau `UnaryOpNode`, dan pemanggilan procedure atau function menjadi `ProcCallNode` atau `FuncCallNode`.

```

1  std::unique_ptr<ProgramNode> buildProgram(const ParseNode& node) {
2      if (node.Label != "<program>") throw std::runtime_error("buildAST expected <program> root");
3
4      auto program = std::make_unique<ProgramNode>();
5      program->Location = locOf(node);
6
7      if (const ParseNode* header = firstChildLabel(node, "<program-header>")) {
8          if (const ParseNode* ident = firstToken(*header, TokenType::IDENT)) program->name = ident->token->value;
9      }
10
11     if (const ParseNode* decl = firstChildLabel(node, "<declaration-part>")) program->declarations = buildDeclarationPart(*decl);
12     if (const ParseNode* body = firstChildLabel(node, "<compound-statement>")) program->body = buildCompoundStatement(*body);
13
14     return program;
15 }
16
17 } // namespace
18
19 std::unique_ptr<ProgramNode> buildAST(const ParseNode& root) {
20     return buildProgram(root);
21 }

```

Gambar Fungsi Utama Transformasi ke AST

Selama proses transformasi, tidak semua node dari parse tree dipertahankan. Node yang dipertahankan mencakup node program dan block, node deklarasi seperti deklarasi variabel, konstanta, tipe, parameter, procedure, dan function, serta node tipe seperti named type, array type, record type, subrange type, dan enumerated type. Selain itu, AST juga mempertahankan node statement, seperti assignment, conditional statement, loop, case statement, repeat statement, dan procedure call. Untuk bagian ekspresi, AST mempertahankan node operasi biner, operasi unary, variabel, literal integer, real, char, string, boolean, function call, array access, dan record access. Dengan pemilihan node tersebut, AST tetap mampu merepresentasikan struktur penting program tanpa membawa seluruh detail sintaks dari parse tree.

Proses pembentukan AST dilakukan dengan sekumpulan fungsi pembantu yang menangani bagian program tertentu. Misalnya, fungsi pembentuk deklarasi membaca bagian `<const-declaration>`, `<type-declaration>`, `<var-declaration>`, dan `<subprogram-declaration>`. Fungsi pembentuk statement menangani assignment, if, while, for, repeat, case, dan procedure call. Sementara itu, fungsi pembentuk ekspresi menangani precedence operator melalui struktur expression, simple expression, term, dan factor. Pendekatan ini membuat transformasi parse tree ke AST tetap mengikuti struktur grammar, tetapi menghasilkan representasi akhir yang lebih sesuai untuk semantic analyzer.

```

1  std::unique_ptr<StatementNode> buildStatement(const ParseNode& node) {
2      if (node.Label == "<statement>" && node.children.empty()) return nullptr;
3      if (node.Label == "<assignment-statement>") return buildAssignment(node);
4      if (node.Label == "<if-statement>") return buildIf(node);
5      if (node.Label == "<while-statement>") return buildWhile(node);
6      if (node.Label == "<for-statement>") return buildFor(node);
7      if (node.Label == "<repeat-statement>") return buildRepeat(node);
8      if (node.Label == "<case-statement>") return buildCase(node);
9      if (node.Label == "<procedure/function-call>") return buildProcCall(node);
10
11     if (node.Label == "<statement>" && !node.children.empty()) {
12         return buildStatement(node.children.front());
13     }
14
15     return nullptr;
16 }
17
18 std::unique_ptr<BlockNode> buildCompoundStatement(const ParseNode& node) {
19     auto block = std::make_unique<BlockNode>();
20     block->location = locOf(node);
21     if (const ParseNode* list = firstChildLabel(node, "<statement-list>")) collectStatements(*list, block->statements);
22     return block;
23 }

```

Gambar Contoh Fungsi Transformasi Parse Node ke AST Node

Setelah AST terbentuk, node-node tersebut masih berisi struktur dasar program tanpa informasi semantik lengkap. Tahap semantic analysis kemudian melakukan traversal terhadap AST untuk mengisi informasi tambahan seperti tipe hasil inferensi, indeks symbol table, dan lexical level.

Secara ringkas, pemetaan Parse Node di milestone 2 ke fungsi pembentukan AST Node dapat dilihat sebagai berikut:

<program>	-> buildAST()
<declaration-part>	
-> buildDeclarationPart()	
<const-declaration>	-> appendConstDecls()
<type-declaration>	-> appendTypeDecls()
<var-declaration>	-> appendVarDecls()
<formal-parameter-list>	->
buildFormalParameters()	
<procedure-declaration>	-> buildProcedure()
<function-declaration>	-> buildFunction()

<type>	-> buildType()	
IDENT sebagai tipe		->
namedTypeFromToken()		
<range>	-> buildRange()	
<enumerated>	-> buildEnumerated()	
<array-type>	-> buildArrayType()	
<record-type>	-> buildRecordType()	
<field-part>	-> buildFieldPart()	
<compound-statement>		->
buildCompoundStatement()		
<statement-list>	-> collectStatements()	
<statement>	-> buildStatement()	
<assignment-statement>	-> buildAssignment()	
<if-statement>	-> buildIf()	
<while-statement>	-> buildWhile()	
<for-statement>	-> buildFor()	
<repeat-statement>	-> buildRepeat()	
<case-statement>	-> buildCase()	
<case-block>		->
collectCaseBranches()		
<procedure/function-call> statement	-> buildProcCall()	
<expression>	-> buildExpression()	
<simple-expression>		->
buildSimpleExpression()		
<term>	-> buildTerm()	

<factor>	-> buildFactor()
<variable>	-> buildVariable()
<component-variable> array/record	-> buildVariable()
<parameter-list>	-> buildArguments()
<procedure/function-call> ekspresi	-> buildFuncCall()
<constant>	-> buildConstant()
intcon	-> literalFromToken()
realcon	-> literalFromToken()
charcon	-> literalFromToken()
string	-> literalFromToken()
ident true/false	-> literalFromToken()
operator unary +, -, not	-> buildConstant() /
buildFactor() / buildSimpleExpression()	
operator biner dan relasional	-> buildExpression() /
buildSimpleExpression() / buildTerm()	

3.5. Perancangan Symbol Table

Symbol table digunakan selama proses semantic analysis untuk melakukan pengecekan deklarasi, lookup identifier, validasi scope, pemeriksaan kompatibilitas tipe, pengecekan parameter procedure/function, serta pemberian anotasi pada AST. Dengan adanya symbol table, semantic analyzer dapat mengetahui apakah suatu identifier sudah dideklarasikan, berada pada scope yang valid, memiliki tipe yang sesuai, dan dapat digunakan dalam konteks tertentu.

Symbol table pada program kami direpresentasikan melalui kelas SymbolTable. Kelas ini mengelola tiga tabel utama, yaitu tab, btab, dan atab. Tabel tab digunakan sebagai tabel utama identifier, btab digunakan untuk menyimpan informasi block atau scope, sedangkan atab digunakan untuk menyimpan informasi tipe array.

```

1 struct TypeInfo {
2     int code = TYPE_VOID;
3     int baseType = TYPE_VOID;
4     int ref = 0;
5     int low = 0;
6     int high = 0;
7     int stringLength = -1;
8     bool isNamed = false;
9     std::string name;
10 };
11
12 struct TabEntry {
13     std::string identifier;
14     int link = 0;
15     int obj = OBJ_VARIABLE;
16     int type = TYPE_VOID;
17     int ref = 0;
18     int nrm = 1;
19     int lev = 0;
20     int adr = 0;
21     TypeInfo typeInfo;
22 };
23
24 struct BTabEntry {
25     int last = 0;
26     int lpar = 0;
27     int psze = 0;
28     int vsze = 0;
29 };
30
31 struct ATabEntry {
32     int xtyp = TYPE_VOID;
33     int etyp = TYPE_VOID;
34     int eref = 0;
35     int low = 0;
36     int high = 0;
37     int elsz = 0;
38     int size = 0;
39 };

```

Gambar Struktur Tiap Symbol Table

Tabel tab menyimpan informasi setiap identifier yang dikenali oleh semantic analyzer. Setiap entry pada tab direpresentasikan sebagai TabEntry yang memuat nama identifier, jenis objek, tipe, referensi, mode parameter, lexical level, alamat relatif, serta link ke entry lain dalam scope yang sama. Jenis objek yang dapat disimpan meliputi constant, variable, type, procedure, function, program, dan reserved word. Tabel ini juga menyimpan predefined identifier dan reserved word seperti tipe dasar Integer, Real, Boolean, Char, String, konstanta True dan False, serta procedure bawaan seperti Read, Readln, Write, dan Writeln.

Tabel btab digunakan untuk merepresentasikan block atau scope. Setiap entry pada btab direpresentasikan sebagai BTabEntry yang menyimpan informasi entry terakhir dalam block, parameter terakhir, ukuran parameter, dan ukuran variabel lokal. Tabel ini digunakan ketika program memasuki scope baru, misalnya pada procedure, function, record, atau main block. Untuk mengelola scope aktif, kelas SymbolTable menggunakan struktur display, sehingga lookup identifier dapat dimulai dari scope terdalam lalu naik ke scope luar.

Tabel atab digunakan untuk menyimpan informasi array. Setiap entry pada atab direpresentasikan sebagai ATabEntry yang memuat tipe indeks, tipe elemen, referensi

elemen, batas bawah, batas atas, ukuran elemen, dan ukuran total array. Informasi ini digunakan untuk memeriksa validitas indeks array, menghitung ukuran array, serta menentukan tipe elemen ketika semantic analyzer menemukan akses array pada ekspresi atau assignment.

Selain menyimpan data, kelas SymbolTable juga menyediakan operasi penting seperti enterTab(), enterBTab(), dan enterATab() untuk menambahkan entry baru ke masing-masing tabel. Fungsi lookupTab() digunakan untuk mencari identifier pada scope aktif dan scope luarnya, sedangkan lookupCurrentScope() digunakan untuk mendeteksi redeklarasasi pada scope yang sama. Fungsi pushScope() dan popScope() digunakan untuk mengatur perpindahan scope saat semantic analyzer memasuki atau keluar dari block tertentu. Dengan rancangan ini, symbol table dapat mendukung kebutuhan analisis semantik secara bertahap dan terstruktur.

```
1 class SymbolTable {
2 public:
3     void init();
4     void clear();
5
6     int enterTab(const TabEntry& entry);
7     int enterBTab(const BTabEntry& entry);
8     int enterATab(const ATabEntry& entry);
9
10    int lookupTab(const std::string& name) const;
11    int lookupTab(const std::string& name, int blockIdx) const;
12    int lookupCurrentScope(const std::string& name) const;
13
14    void pushScope(int btabIdx);
15    void popScope();
16
17    int currentLevel() const;
18    int currentBlock() const;
19
20    TabEntry& tabAt(int index);
21    const TabEntry& tabAt(int index) const;
22    BTabEntry& btabAt(int index);
23    const BTabEntry& btabAt(int index) const;
24    ATabEntry& atabAt(int index);
25    const ATabEntry& atabAt(int index) const;
26
27    const std::vector<TabEntry>& tab() const;
28    const std::vector<BTabEntry>& btab() const;
29    const std::vector<ATabEntry>& atab() const;
30
31    TypeInfo typeOf(int tabIndex) const;
32    std::string typeName(const TypeInfo& type) const;
33    int sizeOf(const TypeInfo& type) const;
34
35 private:
36     void initReservedWords();
37     void initPredefinedIdentifiers();
38
39     std::vector<TabEntry> tabEntries;
40     std::vector<BTabEntry> btabEntries;
41     std::vector<ATabEntry> atabEntries;
42     std::vector<int> display;
43 };
```

Gambar Kelas Symbol Table

3.6. Implementasi Semantic Analyzer

Semantic analyzer pada program kami diimplementasikan melalui kelas `SemanticAnalyzer`. Kelas ini menjadi pengendali utama proses semantic analysis setelah AST berhasil dibentuk. Proses analisis dimulai dari pemanggilan method `analyze()`, yang menerima root parse tree, mengubahnya menjadi AST, lalu menjalankan traversal terhadap AST. Selain itu, tersedia juga method `analyzeAST()` yang dapat langsung menerima root AST berupa `ProgramNode`. Selama proses traversal, semantic analyzer membangun symbol table, melakukan pengecekan aturan semantik, serta menambahkan anotasi tipe, indeks symbol table, dan lexical level pada node AST.

Implementasi semantic analyzer menggunakan pendekatan visitor. Setiap kelompok node AST diproses oleh method yang sesuai dengan jenis node tersebut. Node program dan block diproses melalui `visitProgram()` dan `visitBlock()`. Node deklarasi diproses melalui `visitDeclaration()` yang kemudian meneruskan pemeriksaan ke `visitVarDecl()`, `visitConstDecl()`, `visitTypeDecl()`, `visitParamDecl()`, `visitProcDecl()`, atau `visitFuncDecl()`. Dengan pembagian ini, setiap jenis deklarasi memiliki aturan pemeriksaan yang terpisah dan lebih mudah ditelusuri.



```
1 class SemanticAnalyzer {
2 public:
3     SemanticAnalyzer();
4
5     void analyze(const ParseNode& root);
6     void analyzeAST(std::unique_ptr<ProgramNode> root);
7
8     bool hasErrors() const;
9     const std::vector<SemanticDiagnostic>& errors() const;
10    const std::vector<SemanticDiagnostic>& warnings() const;
11    const ProgramNode* ast() const;
12    const SymbolTable& symbols() const;
13
14    std::vector<std::string> formatOutput() const;
15    std::vector<std::string> formatDecoratedAST() const;
16    std::vector<std::string> formatSymbolTables() const;
17
18 private:
19    std::unique_ptr<ProgramNode> buildASTFromParseTree(const ParseNode& root);
20
21    void visitProgram(ProgramNode* node);
22    void visitBlock(BlockNode* node);
```

Gambar Cuplikan Kelas Semantic Analyzer

Pada analisis deklarasi, semantic analyzer memeriksa apakah suatu identifier sudah pernah dideklarasikan dalam scope yang sama. Jika belum, identifier tersebut dimasukkan ke symbol table melalui `enterTab()`. Deklarasi variabel akan menyimpan tipe, alamat relatif, dan ukuran variabel. Deklarasi konstanta akan mengevaluasi nilai konstanta dan menyimpan tipe serta nilai ordinal jika tersedia. Deklarasi tipe akan menyimpan informasi tipe baru, termasuk named type, subrange, enum, array, dan record. Untuk procedure dan function, analyzer membuat block baru pada `btab`, memasukkan nama routine ke `tab`, lalu memproses parameter, deklarasi lokal, dan body routine pada scope baru.

Analisis tipe dilakukan melalui method `resolveType()` dan method turunannya, seperti `resolveNamedType()`, `resolveArrayType()`, `resolveRecordType()`, `resolveSubrangeType()`, dan `resolveEnumType()`. Pada tahap ini, semantic analyzer memastikan bahwa tipe yang digunakan telah dideklarasikan, batas subrange valid, indeks array menggunakan tipe yang sesuai, serta field record tidak mengalami redeklarasi. Informasi tipe disimpan dalam struktur `TypeInfo`, yang memuat kode tipe, tipe dasar, referensi ke tabel lain, batas bawah dan atas, panjang string, serta nama tipe jika tipe tersebut merupakan named type.

Untuk ekspresi, semantic analyzer menggunakan `visitExpression()` sebagai dispatcher utama. Method ini akan memproses literal, variabel, array access, record access, function call, unary operator, dan binary operator. Pada operasi aritmatika, analyzer memastikan operand bertipe numerik. Pada operasi logika, operand harus bertipe Boolean. Pada operator relasional, analyzer memeriksa kompatibilitas kedua operand dan menghasilkan tipe Boolean. Selain itu, akses array dicek berdasarkan tipe indeks dan batas subrange, sedangkan akses record dicek berdasarkan keberadaan field pada record yang digunakan.

Untuk statement, semantic analyzer menggunakan `visitStatement()` sebagai dispatcher utama. Assignment diproses oleh `visitAssign()` dengan memeriksa apakah sisi kiri dapat di-assign dan apakah tipe nilai kompatibel dengan tipe target. Statement `if`, `while`, dan `repeat` memeriksa bahwa kondisi bertipe Boolean. Statement `for` memeriksa bahwa control variable bertipe ordinal dan nilai awal serta akhir kompatibel dengan tipe control variable. Statement `case` memastikan selector dan label memiliki tipe yang sesuai. Pemanggilan procedure diperiksa melalui `visitProcCall()` dengan mengecek keberadaan procedure, jumlah argumen, tipe argumen, serta aturan parameter by-reference.

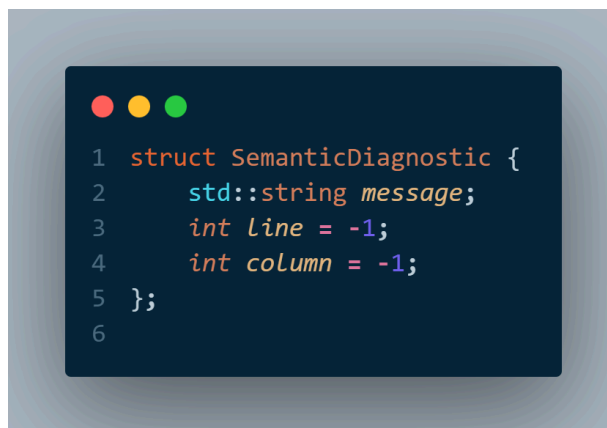
Setelah node selesai dianalisis, semantic analyzer memberi anotasi melalui method `annotate()`. Anotasi tersebut mencakup tipe hasil inferensi, indeks entry pada symbol

table, dan lexical level. Informasi ini kemudian digunakan untuk menghasilkan decorated AST pada output Milestone 3.

3.7. Error Handling dan Diagnostic

Pada program kami, error semantik ditangani melalui mekanisme diagnostics. Kesalahan yang ditemukan selama proses semantic analysis tidak langsung menghentikan eksekusi program pada error pertama. Sebaliknya, semantic analyzer akan mencatat error tersebut ke dalam daftar diagnostic, lalu berusaha melanjutkan analisis pada bagian program berikutnya. Pendekatan ini membuat compiler dapat menampilkan lebih dari satu kesalahan dalam satu kali proses analisis.

Diagnostic direpresentasikan menggunakan struktur SemanticDiagnostic. Struktur ini menyimpan pesan kesalahan serta lokasi source code berupa nomor baris dan kolom. Informasi lokasi diperoleh dari node AST yang sebelumnya dibentuk dari parse tree dan token lexer. Dengan demikian, ketika semantic analyzer menemukan kesalahan seperti identifier yang belum dideklarasikan, redeklarasasi pada scope yang sama, tipe assignment yang tidak sesuai, atau argumen procedure yang salah, pesan error dapat ditampilkan bersama posisi terjadinya kesalahan.



```
1 struct SemanticDiagnostic {
2     std::string message;
3     int line = -1;
4     int column = -1;
5 };
6
```

Gambar Struktur Semantic Diagnostic

Semantic analyzer menyediakan dua method utama untuk mencatat diagnostic, yaitu semanticError() dan semanticWarning(). Method semanticError() digunakan untuk mencatat kesalahan semantik yang membuat program tidak valid, sedangkan semanticWarning() disiapkan untuk peringatan yang tidak langsung menggagalkan proses

analisis. Seluruh error disimpan dalam `errorList`, sementara warning disimpan dalam `warningList`. Status akhir analisis ditentukan berdasarkan keberadaan error pada `errorList`.

Kesalahan semantik yang dideteksi oleh program kami mencakup beberapa kategori. Pada bagian deklarasi, analyzer dapat mendeteksi redeklarasi identifier dalam scope yang sama, penggunaan tipe yang belum dikenal, konstanta yang tidak valid, serta field record yang dideklarasikan ganda. Pada bagian ekspresi, analyzer memeriksa penggunaan identifier yang belum dideklarasikan, penggunaan identifier yang tidak dapat dipakai sebagai nilai, ketidaksesuaian operand operator, akses array dengan indeks yang tidak sesuai, akses record terhadap field yang tidak ada, serta pemanggilan function dengan argumen yang tidak valid. Pada bagian statement, analyzer memeriksa assignment ke target yang tidak dapat diubah, assignment dengan tipe yang tidak kompatibel, kondisi non-Boolean pada control flow, kesalahan selector dan label pada case statement, serta kesalahan pemanggilan procedure.

Output diagnostic ditampilkan pada bagian Semantic diagnostics. Jika tidak ada error maupun warning, compiler akan menampilkan pesan bahwa tidak ada kesalahan semantik. Jika terdapat diagnostic, setiap warning atau error ditampilkan dengan format yang memuat jenis diagnostic, lokasi baris dan kolom jika tersedia, serta pesan kesalahan. Setelah seluruh diagnostic dicetak, compiler menampilkan status akhir berupa SUCCESS jika tidak ada error, atau FAILED jika terdapat satu atau lebih error.

BAB IV

PENGUJIAN

4.1. Pengujian 1

Input
<pre>test/milestone-3/input/1.txt program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>
Output
<pre>test/milestone-3/output/1.txt tab (relevant entries only): idx id obj type ref nrm lev adr link ----- ... (reserved words 0-32) 33 Hello program 0 0 1 0 0 0 34 a variable 1 0 1 0 0 0 35 b variable 1 0 1 0 1 34 36 writeln procedure 0 0 1 0 0 0 btab: idx last lpar psze vsze ----- 0 35 0 0 2 1 0 0 0 0 atab: (empty; no arrays declared) Decorated AST: Program(Hello, block:0) -> tab_index:33, type:void, lev:0</pre>

```

└─ Declarations
|   └─ VarDecl(a)                -> tab_index:34, type:integer, lev:0
|   └─ VarDecl(b)                -> tab_index:35, type:integer, lev:0
└─ Block(block:1)                -> type:void, lev:1
    └─ Assign                    -> type:void, lev:1
        └─ Target Var(a)         -> tab_index:34, type:integer, lev:0
            └─ Value Int(5)      -> type:integer, lev:1
        └─ Assign                -> type:void, lev:1
            └─ Target Var(b)     -> tab_index:35, type:integer, lev:0
                └─ Value BinOp(+) -> type:integer, lev:1
                    └─ Left Var(a) -> tab_index:34, type:integer, lev:0
                        └─ Right Int(10) -> type:integer, lev:1
            └─ ProcCall(writeln) -> tab_index:36, type:void, lev:0
                └─ Arg String("Result = ") -> type:string, lev:1
                    └─ Arg Var(b) -> tab_index:35, type:integer, lev:0

```

Semantic diagnostics:
No semantic errors or warnings.

Status: SUCCESS

Penjelasan

Testcase ini menguji program valid paling dasar yang memiliki deklarasi variabel, assignment, ekspresi aritmetika, dan pemanggilan prosedur writeln. Output semantic analyzer menunjukkan bahwa program berhasil membentuk tab untuk program, variabel a dan b, serta prosedur bawaan writeln. Pada Decorated AST, setiap deklarasi dan penggunaan identifier sudah memiliki anotasi tab_index, type, dan lev.

Output juga menunjukkan bahwa semantic analyzer berhasil melakukan type checking pada assignment a := 5 dan b := a + 10. Ekspresi a + 10 dikenali sebagai BinOp(+) bertipe integer, sehingga assignment ke variabel b valid. Diagnostics menghasilkan No semantic errors or warnings dan status SUCCESS, sehingga testcase ini membuktikan bahwa program mampu menangani kasus valid dasar sesuai spesifikasi Milestone 3.

4.2. Pengujian 2

Input

```

test/milestone-3/input/2.txt

program Declarations;

const
    Limit == 10;
    Letter == 'z';

```

```

    Title == 'ok';

type
    Index == 1..10;
    Day == (Mon, Tue, Wed);
    Scores == array[Index] of integer;
    Person == record
        age: integer;
        grade: char;
    end;

var
    i: Index;
    today: Day;
    values: Scores;
    p: Person;

begin
    i := Limit;
    today := Tue;
    values[i] := Limit;
    p.age := values[1];
    p.grade := Letter;
    writeln(Title);
end.

```

Output

test/milestone-3/output/2.txt

tab (relevant entries only):

idx	id	obj	type	ref	nrm	lev	adr	link

...	(reserved words 0-32)							
33	Declarations	program	0	0	1	0	0	0
34	Limit	constant	1	0	1	0	10	0
35	Letter	constant	3	0	1	0	122	34
36	Title	constant	5	0	1	0	0	35
37	Index	type	8	0	1	0	0	0
38	Day	type	9	0	1	0	0	37
39	Mon	constant	9	0	1	0	0	0
40	Tue	constant	9	0	1	0	1	39
41	Wed	constant	9	0	1	0	2	40
42	Scores	type	6	1	1	0	0	0
43	age	variable	1	0	1	1	0	0
44	grade	variable	3	0	1	1	1	43

45	Person	type	7	1	1	0	0	42
46	i	variable	8	0	1	0	0	0
47	today	variable	9	0	1	0	1	46
48	values	variable	6	1	1	0	2	47
49	p	variable	7	1	1	0	12	48
50	writeln	procedure	0	0	1	0	0	0

btab:

idx	last	lpar	psze	vsze

0	49	0	0	14
1	44	0	0	2
2	0	0	0	0

atab:

idx	xtyp	etyp	eref	low	high	elsz	size

1	1	1	0	1	10	1	10

Decorated AST:

```

Program(Declarations, block:0)                                -> tab_index:33, type:void, lev:0
├─ Declarations
│   ├─ ConstDecl(Limit)                                       -> tab_index:34, type:integer, lev:0
│   │   └─ Value Int(10)                                       -> tab_index:34, type:integer, lev:0
│   ├─ ConstDecl(Letter)                                       -> tab_index:35, type:char, lev:0
│   │   └─ Value Char('z')                                     -> tab_index:35, type:char, lev:0
│   ├─ ConstDecl(Title)                                        -> tab_index:36, type:string, lev:0
│   │   └─ Value String("ok")                                  -> tab_index:36, type:string, lev:0
│   ├─ TypeDecl(Index)                                         -> tab_index:37, type:subrange, lev:0
│   │   └─ Type SubrangeType                                   -> tab_index:37, type:subrange, lev:0
│   │       └─ Lower Int(1)                                    -> type:integer, lev:0
│   │           └─ Upper Int(10)                               -> type:integer, lev:0
│   ├─ TypeDecl(Day)                                           -> tab_index:38, type:enum, lev:0
│   │   └─ Type EnumType(Mon, Tue, Wed)                       -> tab_index:38, type:enum, lev:0
│   ├─ TypeDecl(Scores)                                        -> tab_index:42, type:array, lev:0
│   │   └─ Type ArrayType                                       -> tab_index:42, type:array, lev:0
│   │       └─ IndexType NamedType(Index)                     -> tab_index:37, type:subrange, lev:0
│   │           └─ ElementType NamedType(integer)              -> type:integer, lev:0
│   ├─ TypeDecl(Person)                                        -> tab_index:45, type:record, lev:0
│   │   └─ Type RecordType                                     -> tab_index:45, type:record, lev:0
│   │       └─ Field(age) NamedType(integer)                   -> type:integer, lev:0
│   │           └─ Field(grade) NamedType(char)                -> type:char, lev:0
│   ├─ VarDecl(i)                                              -> tab_index:46, type:subrange, lev:0
│   ├─ VarDecl(today)                                         -> tab_index:47, type:enum, lev:0
│   ├─ VarDecl(values)                                         -> tab_index:48, type:array, lev:0
│   └─ VarDecl(p)                                              -> tab_index:49, type:record, lev:0

```

└ Block(block:2)	-> type:void, lev:1
├ Assign	-> type:void, lev:1
│ └ Target Var(i)	-> tab_index:46, type:subrange, lev:0
│ └ Value Var(Limit)	-> tab_index:34, type:integer, lev:0
├ Assign	-> type:void, lev:1
│ └ Target Var(today)	-> tab_index:47, type:enum, lev:0
│ └ Value Var(Tue)	-> tab_index:40, type:enum, lev:0
├ Assign	-> type:void, lev:1
│ └ Target ArrayAccess	-> tab_index:48, type:integer, lev:0
│ │ └ Array Var(values)	-> tab_index:48, type:array, lev:0
│ │ └ Index Var(i)	-> tab_index:46, type:subrange, lev:0
│ └ Value Var(Limit)	-> tab_index:34, type:integer, lev:0
├ Assign	-> type:void, lev:1
│ └ Target RecordAccess(age)	-> tab_index:43, type:integer, lev:1
│ │ └ Record Var(p)	-> tab_index:49, type:record, lev:0
│ └ Value ArrayAccess	-> tab_index:48, type:integer, lev:0
│ │ └ Array Var(values)	-> tab_index:48, type:array, lev:0
│ │ └ Index Int(1)	-> type:integer, lev:1
├ Assign	-> type:void, lev:1
│ └ Target RecordAccess(grade)	-> tab_index:44, type:char, lev:1
│ │ └ Record Var(p)	-> tab_index:49, type:record, lev:0
│ └ Value Var(Letter)	-> tab_index:35, type:char, lev:0
└ ProcCall(writeln)	-> tab_index:50, type:void, lev:0
└ Arg Var(Title)	-> tab_index:36, type:string, lev:0

Semantic diagnostics:
No semantic errors or warnings.

Status: SUCCESS

Penjelasan

Testcase ini menguji deklarasi yang lebih lengkap, meliputi `const`, `type`, `subrange`, `enumerated type`, `array`, `record`, serta variabel yang menggunakan tipe-tipe tersebut. Output `tab`, `btabs`, dan `atab` menunjukkan bahwa semantic analyzer berhasil menyimpan informasi identifier, scope, ukuran variabel, field record, dan struktur array. Keberadaan `atab` membuktikan bahwa tipe array berhasil direpresentasikan beserta index type, element type, batas bawah, batas atas, dan ukuran elemen.

Decorated AST juga memperlihatkan bahwa setiap deklarasi tipe dan variabel diberi anotasi semantik yang sesuai. Assignment seperti `values[i] := Limit`, `p.age := values[1]`, dan `p.grade := Letter` menunjukkan bahwa program mampu memeriksa array access, record access, dan kompatibilitas tipe. Karena output berakhir dengan `Status: SUCCESS`, testcase ini membuktikan bahwa program memenuhi spesifikasi untuk symbol table management dan type checking pada tipe data kompleks.

4.3. Pengujian 3

Input
<pre>test/milestone-3/input/3.txt program ControlFlow; var i, total: integer; ok: boolean; begin total := 0; ok := True; if ok then total := total + 1 else total := total - 1; while total < 5 do begin total := total + 1 end; repeat total := total - 1 until total < 1; for i := 1 to 3 do begin total := total + i; end; end. end.</pre>
Output
<pre>test/milestone-3/output/3.txt tab (relevant entries only): idx id obj type ref nrm lev adr link -----</pre>

```

... (reserved words 0-32)
33 ControlFlow program      0  0  1  0  0  0
34 i      variable         1  0  1  0  0  0
35 total   variable         1  0  1  0  1 34
36 ok      variable         4  0  1  0  2 35

btab:
idx  last  lpar  psze  vsze
-----
0     36    0    0    3
1      0    0    0    0

atab: (empty; no arrays declared)

Decorated AST:
Program(ControlFlow, block:0)      -> tab_index:33, type:void, lev:0
├─ Declarations
│   ├─ VarDecl(i)                  -> tab_index:34, type:integer, lev:0
│   ├─ VarDecl(total)              -> tab_index:35, type:integer, lev:0
│   └─ VarDecl(ok)                  -> tab_index:36, type:boolean, lev:0
└─ Block(block:1)                  -> type:void, lev:1
    ├─ Assign                      -> type:void, lev:1
    │   ├─ Target Var(total)        -> tab_index:35, type:integer, lev:0
    │   └─ Value Int(0)              -> type:integer, lev:1
    ├─ Assign                      -> type:void, lev:1
    │   ├─ Target Var(ok)           -> tab_index:36, type:boolean, lev:0
    │   └─ Value Bool(true)         -> type:boolean, lev:1
    ├─ If                          -> type:void, lev:1
    │   ├─ Condition Var(ok)        -> tab_index:36, type:boolean, lev:0
    │   ├─ Then Assign              -> type:void, lev:1
    │   │   ├─ Target Var(total)    -> tab_index:35, type:integer, lev:0
    │   │   └─ Value BinOp(+)       -> type:integer, lev:1
    │   │       ├─ Left Var(total)   -> tab_index:35, type:integer, lev:0
    │   │       └─ Right Int(1)     -> type:integer, lev:1
    │   └─ Else Assign              -> type:void, lev:1
    │       ├─ Target Var(total)    -> tab_index:35, type:integer, lev:0
    │       └─ Value BinOp(-)       -> type:integer, lev:1
    │           ├─ Left Var(total)   -> tab_index:35, type:integer, lev:0
    │           └─ Right Int(1)     -> type:integer, lev:1
    ├─ While                      -> type:void, lev:1
    │   ├─ Condition BinOp(<)       -> type:boolean, lev:1
    │   │   ├─ Left Var(total)     -> tab_index:35, type:integer, lev:0
    │   │   └─ Right Int(5)        -> type:integer, lev:1
    │   └─ Body Block(block:1)     -> type:void, lev:1
    │       └─ Assign              -> type:void, lev:1
    │           └─ Target Var(total) -> tab_index:35, type:integer, lev:0

```

```

|           └─ Value BinOp(+)      -> type:integer, lev:1
|           └─ Left Var(total)    -> tab_index:35, type:integer, lev:0
|           └─ Right Int(1)       -> type:integer, lev:1
└─ Repeat                          -> type:void, lev:1
    └─ Body Assign                  -> type:void, lev:1
        └─ Target Var(total)       -> tab_index:35, type:integer, lev:0
            └─ Value BinOp(-)      -> type:integer, lev:1
                └─ Left Var(total)  -> tab_index:35, type:integer, lev:0
                    └─ Right Int(1) -> type:integer, lev:1
└─ Until BinOp(<)                 -> type:boolean, lev:1
    └─ Left Var(total)            -> tab_index:35, type:integer, lev:0
        └─ Right Int(1)          -> type:integer, lev:1
└─ For(i, to)                     -> type:void, lev:1
    └─ Start Int(1)               -> type:integer, lev:1
    └─ End Int(3)                 -> type:integer, lev:1
    └─ Body Block(block:1)        -> type:void, lev:1
        └─ Assign                  -> type:void, lev:1
            └─ Target Var(total)   -> tab_index:35, type:integer, lev:0
                └─ Value BinOp(+)  -> type:integer, lev:1
                    └─ Left Var(total) -> tab_index:35, type:integer, lev:0
                        └─ Right Var(i) -> tab_index:34, type:integer, lev:0

```

Semantic diagnostics:

No semantic errors or warnings.

Status: SUCCESS

Penjelasan

Testcase ini menguji semantic analysis pada control flow, yaitu if-else, while, repeat-until, dan for. Output Decorated AST menunjukkan bahwa setiap struktur kontrol berhasil dibentuk sebagai node seperti If, While, Repeat, dan For, dengan child seperti Condition, Then, Else, Body, Start, dan End. Setiap ekspresi kondisi juga dianotasi dengan tipe yang sesuai, misalnya hasil perbandingan bertipe boolean.

Testcase ini membuktikan bahwa program tidak hanya mengecek deklarasi dan assignment sederhana, tetapi juga mampu melakukan analisis semantik pada alur kontrol. Assignment di dalam setiap blok tetap diperiksa tipenya, dan variabel yang digunakan di dalam loop tetap terhubung ke entry symbol table yang benar. Output SUCCESS menunjukkan bahwa semua struktur kontrol valid secara semantik.

4.4. Pengujian 4

Input									
<pre> test/milestone-3/input/5.txt program FunctionValid; var z: integer; function Max(a, b: integer): integer; begin if a > b then Max := a else Max := b; end; begin z := Max(3, 5); writeln(z); end. </pre>									
Output									
<pre> test/milestone-3/output/5.txt tab (relevant entries only): idx id obj type ref nrm lev adr link ----- ... (reserved words 0-32) 33 FunctionValid program 0 0 1 0 0 0 34 z variable 1 0 1 0 0 0 35 Max function 1 1 1 0 0 0 36 a variable 1 0 1 1 0 0 37 b variable 1 0 1 1 1 36 38 writeln procedure 0 0 1 0 0 0 btab: idx last lpar psze vsze ----- 0 35 0 0 1 </pre>									

```

1      37    37    2    0
2      0     0     0     0

```

atab: (empty; no arrays declared)

Decorated AST:

```

Program(FunctionValid, block:0)          -> tab_index:33, type:void, lev:0
├─ Declarations
│   ├─ VarDecl(z)                        -> tab_index:34, type:integer, lev:0
│   └─ FuncDecl(Max, block:1)            -> tab_index:35, type:integer, lev:0
│       ├─ Parameters
│       │   ├─ ParamDecl(a)              -> tab_index:36, type:integer, lev:1
│       │   └─ ParamDecl(b)              -> tab_index:37, type:integer, lev:1
│       ├─ ReturnType NamedType(integer) -> type:integer, lev:0
│       └─ Body Block(block:1)            -> type:void, lev:1
│           └─ If
│               ├─ Condition BinOp(>)    -> type:boolean, lev:1
│               │   ├─ Left Var(a)        -> tab_index:36, type:integer, lev:1
│               │   └─ Right Var(b)       -> tab_index:37, type:integer, lev:1
│               └─ Then Assign
│                   ├─ Target Var(Max)     -> tab_index:35, type:integer, lev:0
│                   └─ Value Var(a)        -> tab_index:36, type:integer, lev:1
│                       └─ Else Assign
│                           ├─ Target Var(Max) -> tab_index:35, type:integer, lev:0
│                           └─ Value Var(b)  -> tab_index:37, type:integer, lev:1
└─ Block(block:2)                        -> type:void, lev:1
    ├─ Assign                            -> type:void, lev:1
    │   ├─ Target Var(z)                  -> tab_index:34, type:integer, lev:0
    │   └─ Value FuncCall(Max)            -> tab_index:35, type:integer, lev:0
    │       ├─ Arg Int(3)                  -> type:integer, lev:1
    │       └─ Arg Int(5)                  -> type:integer, lev:1
    └─ ProcCall(writeln)                  -> tab_index:38, type:void, lev:0
        └─ Arg Var(z)                      -> tab_index:34, type:integer, lev:0

```

Semantic diagnostics:

No semantic errors or warnings.

Status: SUCCESS

Penjelasan

Testcase ini menguji deklarasi dan pemanggilan fungsi. Fungsi Max memiliki dua parameter bertipe integer dan return type integer. Output tab menunjukkan bahwa fungsi, parameter, variabel lokal/global, dan prosedur writeln berhasil masuk ke symbol table dengan scope dan lexical level yang tepat. Decorated AST juga menampilkan FuncDecl(Max), ReturnType, parameter, serta body fungsi.

Pada bagian utama program, pemanggilan `Max(3, 5)` dianotasi sebagai `FuncCall(Max)` bertipe integer, sehingga assignment ke variabel `z` valid. Di dalam fungsi, assignment ke nama fungsi `Max := a` atau `Max := b` juga dikenali sebagai mekanisme pengembalian nilai fungsi. Testcase ini membuktikan bahwa semantic analyzer mampu menangani fungsi, parameter, return type, function call, dan scope resolution sesuai kebutuhan Milestone 3.

4.5. Pengujian 5

Input
<pre>test/milestone-3/input/6.txt program TypeMismatch; var x: integer; s: string; begin x := 'oops'; s := 1; end.</pre>
Output
<pre>test/milestone-3/output/6.txt tab (relevant entries only): idx id obj type ref nrm lev adr link ----- ... (reserved words 0-32) 33 TypeMismatch program 0 0 1 0 0 0 34 x variable 1 0 1 0 0 0 35 s variable 5 0 1 0 1 34 btab: idx last lpar psze vsze ----- 0 35 0 0 2 1 0 0 0 0 atab: (empty; no arrays declared)</pre>

```

Decorated AST:
Program(TypeMismatch, block:0) -> tab_index:33, type:void, lev:0
├─ Declarations
│   ├── VarDecl(x) -> tab_index:34, type:integer, lev:0
│   └─ VarDecl(s) -> tab_index:35, type:string, lev:0
└─ Block(block:1) -> type:void, lev:1
    ├── Assign -> type:void, lev:1
    │   ├── Target Var(x) -> tab_index:34, type:integer, lev:0
    │   └─ Value String("oops") -> type:string, lev:1
    └─ Assign -> type:void, lev:1
        ├── Target Var(s) -> tab_index:35, type:string, lev:0
        └─ Value Int(1) -> type:integer, lev:1

Semantic diagnostics:
Error at line 8, column 5: cannot assign String to Integer
Error at line 9, column 5: cannot assign Integer to String

Status: FAILED

```

Penjelasan

Testcase ini menguji error type mismatch pada assignment. Program mencoba mengisi variabel integer `x` dengan string dan variabel string `s` dengan integer. Output Decorated AST tetap dibentuk, tetapi diagnostics menghasilkan error yang menjelaskan bahwa `String` tidak dapat di-assign ke `Integer`, dan `Integer` tidak dapat di-assign ke `String`.

Hal ini menunjukkan bahwa program mampu mendeteksi ketidaksesuaian tipe tanpa crash. Status akhir `FAILED` sesuai dengan tujuan testcase, karena input memang dirancang tidak valid secara semantik. Testcase ini membuktikan bahwa type checking assignment sudah berjalan dan error handling menghasilkan pesan yang informatif.

4.6. Pengujian 6

Input

```

test/milestone-3/input/7.txt

program UndeclaredIdentifier;

begin
    x := 1;
    writeln(y);
end.

```

Output

test/milestone-3/output/7.txt

tab (relevant entries only):

idx	id	obj	type	ref	nrm	lev	adr	link

...	(reserved words 0-32)							
33	UndeclaredIdentifier	program		0	0	1	0	0
34	writeln	procedure		0	0	1	0	0

btab:

idx	last	lpar	psze	vsze

0	33	0	0	0
1	0	0	0	0

atab: (empty; no arrays declared)

Decorated AST:

```
Program(UndeclaredIdentifier, block:0) -> tab_index:33, type:void, lev:0
└─ Block(block:1) -> type:void, lev:1
    │   └─ Assign -> type:void, lev:1
    │       │   └─ Target Var(x) -> type:error, lev:1
    │       │       └─ Value Int(1) -> type:integer, lev:1
    │       └─ ProcCall(writeln) -> tab_index:34, type:void, lev:0
    │           └─ Arg Var(y) -> type:error, lev:1
```

Semantic diagnostics:

Error at line 4, column 5: undeclared identifier 'x'

Error at line 5, column 13: undeclared identifier 'y'

Status: FAILED

Penjelasan

Testcase ini menguji penggunaan identifier yang belum dideklarasikan. Program menggunakan x pada assignment dan y sebagai argumen `writeln`, tetapi keduanya tidak ada di bagian deklarasi. Output `tab` hanya berisi program dan prosedur bawaan yang relevan, sedangkan Decorated AST menandai `Var(x)` dan `Var(y)` dengan `type:error`.

Diagnostics menampilkan error untuk identifier x dan y yang belum dideklarasikan. Ini membuktikan bahwa semantic analyzer melakukan lookup identifier ke symbol table dan mampu membedakan identifier yang valid dan tidak valid. Status `FAILED` menunjukkan bahwa program berhasil menolak input yang melanggar aturan deklarasi.

4.7. Pengujian 7

Input
<pre>test/milestone-3/input/10.txt program ArrayBounds; type Index == 1..3; Values == array[Index] of integer; var v: Values; begin v[4] := 10; end.</pre>
Output
<pre>test/milestone-3/output/10.txt tab (relevant entries only): idx id obj type ref nrm lev adr link ----- ... (reserved words 0-32) 33 ArrayBounds program 0 0 1 0 0 0 34 Index type 8 0 1 0 0 0 35 Values type 6 1 1 0 0 34 36 v variable 6 1 1 0 0 0 btab: idx last lpar psze vsze ----- 0 36 0 0 3 1 0 0 0 0 atab: idx xtyp etyp eref low high elsz size ----- 1 1 1 0 1 3 1 3 Decorated AST:</pre>

<pre> Program(ArrayBounds, block:0) ├─ Declarations ├─ TypeDecl(Index) └─ Type SubrangeType └─ Lower Int(1) └─ Upper Int(3) └─ TypeDecl(Values) └─ Type ArrayType └─ IndexType NamedType(Index) └─ ElementType NamedType(integer) └─ VarDecl(v) └─ Block(block:1) └─ Assign └─ Target ArrayAccess └─ Array Var(v) └─ Index Int(4) └─ Value Int(10) </pre>	<pre> -> tab_index:33, type:void, lev:0 -> tab_index:34, type:subrange, lev:0 -> tab_index:34, type:subrange, lev:0 -> type:integer, lev:0 -> type:integer, lev:0 -> tab_index:35, type:array, lev:0 -> tab_index:35, type:array, lev:0 -> tab_index:34, type:subrange, lev:0 -> type:integer, lev:0 -> tab_index:36, type:array, lev:0 -> type:void, lev:1 -> type:void, lev:1 -> tab_index:36, type:integer, lev:0 -> tab_index:36, type:array, lev:0 -> type:integer, lev:1 -> type:integer, lev:1 </pre>
Semantic diagnostics: Error at line 11, column 7: array index is outside the declared subrange Status: FAILED	
Penjelasan	
Testcase ini menguji pemeriksaan batas array. Program mendeklarasikan tipe indeks 1..3, lalu membuat array dengan indeks tersebut. Pada body program, assignment dilakukan ke <code>v[4]</code>, yaitu indeks yang berada di luar subrange yang dideklarasikan. Output atab menunjukkan bahwa array memiliki batas <code>low = 1</code> dan <code>high = 3</code>. Decorated AST memperlihatkan <code>ArrayAccess</code> pada target assignment, dengan index berupa <code>Int(4)</code>. Diagnostics menghasilkan error bahwa indeks array berada di luar subrange yang dideklarasikan. Testcase ini membuktikan bahwa program tidak hanya mengenali struktur array, tetapi juga mampu melakukan validasi semantik terhadap batas indeks array.	

4.8. Pengujian 8

Input
<pre> test/milestone-3/input/13.txt program ByReferenceInvalid; procedure Reset(var target: integer); begin </pre>

```

    target := 0;
end;

begin
    Reset(1);
end.

```

Output

test/milestone-3/output/13.txt

tab (relevant entries only):

idx	id	obj	type	ref	nrm	lev	adr	link

... (reserved words 0-32)								
33	ByReferenceInvalid	program		0	0	1	0	0
34	Reset	procedure		0	1	1	0	0
35	target	variable		1	0	0	1	0

btab:

idx	last	lpar	psze	vsze

0	34	0	0	0
1	35	35	1	0
2	0	0	0	0

atab: (empty; no arrays declared)

Decorated AST:

```

Program(ByReferenceInvalid, block:0) -> tab_index:33, type:void, lev:0
├─ Declarations
│   └─ ProcDecl(Reset, block:1)      -> tab_index:34, type:void, lev:0
│       └─ Parameters
│           └─ ParamDecl(target, var) -> tab_index:35, type:integer, lev:1
│               └─ Body Block(block:1) -> type:void, lev:1
│                   └─ Assign          -> type:void, lev:1
│                       └─ Target Var(target) -> tab_index:35, type:integer, lev:1
│                           └─ Value Int(0)   -> type:integer, lev:1
└─ Block(block:2)                    -> type:void, lev:1
    └─ ProcCall(Reset)                -> tab_index:34, type:void, lev:0
        └─ Arg Int(1)                 -> type:integer, lev:1

```

Semantic diagnostics:

Error at line 9, column 11: argument 1 of procedure 'Reset' must be assignable for by-reference parameter

Status: FAILED

Penjelasan

Testcase ini menguji parameter by-reference yang tidak valid. Prosedur `Reset` menerima parameter `var target : integer`, sehingga argumen yang diberikan harus berupa target yang assignable, misalnya variabel. Namun program memanggil `Reset (1)`, yaitu literal integer, bukan variabel.

Output Decorated AST menunjukkan bahwa prosedur dan parameternya berhasil dimasukkan ke symbol table, serta pemanggilan `Reset` berhasil dianalisis. Diagnostics kemudian menghasilkan error bahwa argumen pertama harus assignable untuk parameter by-reference. Testcase ini membuktikan bahwa semantic analyzer mampu membedakan parameter by-value dan by-reference, serta menolak argumen yang tidak dapat diberi nilai.

BAB V

KESIMPULAN DAN SARAN

5.1. Kesimpulan

Pada milestone 3 ini, program Arion Compiler telah berhasil dikembangkan dari tahap syntax analysis menuju semantic analysis. Program tidak hanya mampu membaca source code Arion, melakukan lexical analysis, dan membentuk parse tree melalui parser, tetapi juga mampu mengubah parse tree tersebut menjadi Abstract Syntax Tree (AST) yang lebih ringkas untuk kebutuhan analisis semantik.

Semantic analyzer yang dibuat telah mampu melakukan pengecekan makna program menggunakan pendekatan visitor dan symbol table. Program dapat melakukan registrasi dan lookup identifier, mengelola scope, memeriksa deklarasi ulang, memvalidasi type compatibility pada assignment dan ekspresi, serta menangani penggunaan procedure, function, parameter, array, record, dan struktur kontrol. Symbol table yang digunakan juga telah direpresentasikan melalui tab, btab, dan atab sehingga informasi identifier, block, scope, dan array dapat disimpan dengan lebih terstruktur.

Hasil dari proses semantic analysis direpresentasikan dalam bentuk decorated AST, yaitu AST yang telah diberi anotasi berupa tipe data, indeks symbol table, dan lexical level. Selain itu, program juga dapat menghasilkan semantic diagnostics untuk menampilkan pesan kesalahan apabila ditemukan pelanggaran semantik, seperti identifier yang belum dideklarasikan, assignment dengan tipe yang tidak sesuai, indeks array di luar batas, atau argumen by-reference yang tidak valid.

Berdasarkan hasil pengujian, program dapat menangani berbagai kasus valid maupun tidak valid. Untuk input yang valid, program dapat menghasilkan symbol table, decorated AST, dan status SUCCESS. Sementara itu, untuk input yang tidak valid, program dapat memberikan pesan kesalahan yang informatif dan menghasilkan status FAILED tanpa menyebabkan program berhenti secara tidak terkendali. Dengan demikian, program telah memenuhi tujuan milestone 3, yaitu melakukan analisis semantik terhadap source code Arion sesuai spesifikasi yang diberikan.

5.2. Saran

Pelaksanaan Milestone 3 Tugas Besar Teori Bahasa Formal dan Automata di Semester II (Genap) Tahun 2025/2026 merupakan pengalaman yang sangat berharga bagi kami. Dari pengalaman ini, kami ingin berbagi beberapa saran kepada pembaca yang mungkin akan menghadapi tugas serupa di masa depan.

Untuk pengembangan selanjutnya, semantic analyzer dapat dilengkapi dengan pemeriksaan semantik yang lebih lengkap, seperti validasi return pada function, deteksi unreachable code, dan pengecekan control flow yang lebih rinci. Dengan demikian, compiler dapat mendeteksi kesalahan program secara lebih menyeluruh.

Selain itu, pengujian dapat diperluas dengan lebih banyak variasi kasus, terutama untuk nested scope, procedure/function, array, record, dan type compatibility. Decorated AST dan symbol table yang telah dihasilkan juga dapat dikembangkan lebih lanjut agar lebih mudah digunakan pada tahap compiler berikutnya. Tidak lupa, kami ingin mengucapkan terima kasih untuk perpanjangan deadline.

BAB VI

LAMPIRAN

6.1. Lampiran Link Repository Github.

Link : <https://github.com/ucupsss/SMG-Tubes-IF2224-2026>

6.2. Pembagian Tugas

NIM	Nama	Tugas	Kontribusi
13524014	Yusuf Faishal Listyardi	Symbol Table, Semantic Declaration, dan Laporan	25%
13524046	Farrell Limjaya	Semantic Expression, Semantic Statement, dan Laporan	25%
13524066	Nathanael Gunawan	Implementasi Statement, Integrasi Statement, CLI, dan Laporan	25%
13524070	A. Fawwaz Azam Wicaksono	Revisi Milestone 2, Implementasi AST, Test Case, dan Laporan	25%

REFERENSI

- <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0264326#pone.0264326.ref001>
- <https://opensa.cs.vt.edu/ODSA/Books/PL/html/Grammars1.html>
- <https://www.cs.odu.edu/~toida/nerzic/390teched/cfl/cfg.html>
- <https://www.geeksforgeeks.org/compiler-design/semantic-analysis-in-compiler-design/>
- <https://www.cs.toronto.edu/~david/course-notes/csc110-111/16-abstract-syntax-trees/01-abstract-syntax-trees.html>
- <https://www.geeksforgeeks.org/compiler-design/syntax-directed-translation-in-compiler-design/>
- https://www.tutorialspoint.com/compiler_design/compiler_design_attributed_grammars.htm
- <https://www.geeksforgeeks.org/compiler-design/symbol-table-compiler/>
- <https://www.geeksforgeeks.org/compiler-design/type-checking-in-compiler-design/>